Vue3 Compiler

Vue.js 的模板编译器用于把模板编译为渲染函数。它的工作流程大致分为三个步骤。

- 分析模板，将其解析为模板 AST
- 将模板 AST 转换为用于描述渲染函数的 JavaScript AST。
- 根据 JavaScript AST 生成渲染函数代码。

```
const ast = parse(`<div><p>Vue</p><p>Template</p></div>`)
transform(ast)
const code = generate(ast.jsNode)
```

最终得到的代码字符串如下

```
function render() {
  return h('div', [h('p', 'Vue'), h('p', 'Template')])
}
```

Vue3 在模板编译这块也有一定的优化

1. Block Tree 和 PatchFlags

- 传统 `Diff` 算法的问题
- `Block` 配合 `PatchFlags` 做到靶向更新
- 节点不稳定 - `Block Tree`
- `v-if` 的元素作为 `Block`
- `v-for` 的元素作为 `Block`
- 不稳定的 `Fragment`
- 稳定的 `Fragment`
  - `v-for` 的表达式是常量
  - 多个根元素
  - 插槽出口
  - `<template v-for>`

传统 Diff 算法的问题

“传统 `vdom`” 的 `Diff` 算法总归要按照 `vdom` 树的层级结构一层一层的遍历

```
<div>
  <p class="foo">bar</p>
</div>
```

对于传统 `diff` 算法来说，它在 `diff` 这段 `vnode` (模板编译后的 `vnode`)时会经历：

- Div 标签的属性 + children
  - <p> 标签的属性(class) + children
    - 文本节点：bar

但是很明显，这明明就是一段静态 `vdom` ，它在组件更新阶段是不可能发生变化的。如果能在 `diff` 阶段跳过静态内容，那就会避免无用的 `vdom` 树的遍历和比对，这应该就是最早的优化思路来源----**跳过静态内容，只对比动态内容。**

Block 配合 PatchFlags 做到靶向更新

咱们先说 `Block` 再聊 `Block Tree` 。现在思路有了，我们只希望对比非静态的内容，例如：

```
<div>
  <p>foo</p>
  <p>{{ bar }}</p>
</div>
```

在这段模板中，只有 `<p>{{ bar }}</p>` 中的文本节点是动态的，因此只需要靶向更新该文本节点即可，这在包含大量静态内容而只有少量动态内容的场景下，性能优势尤其明显。

这段模板对应的传统 `vdom` 树大概长什么样：

```
const vnode = {
  tag: 'div',
  children: [
    { tag: 'p', children: 'foo' },
    { tag: 'p', children: ctx.bar },  // 这是动态节点
  ]
}
```

在传统的 `vdom` 树中，我们在运行时得不到任何有用信息，但是 `Vue3` 的 `compiler` 能够分析模板并提取有用信息，最终体现在 `vdom` 树上。例如它能够清楚的知道：哪些节点是动态节点，以及为

什么它是动态的(是绑定了动态的 `class`？还是绑定了动态的 `style`？亦或是其它动态的属性？)，总之编译器能够提取我们想要的信息，有了这些信息我们就可以在创建 `vnode` 的过程中为动态的节点打上标记：也就是传说中的 `PatchFlags`。

我们可以把 `PatchFlags` 简单的理解为一个数字标记，把这些数字赋予不同含义，例如：

- 数字 1：代表节点有动态的 `textContent`（例如上面模板中的 `p` 标签）
- 数字 2：代表元素有动态的 `class` 绑定
- 数字 3：代表xxxxx

```
const vnode = {
  tag: 'div',
  children: [
    { tag: 'p', children: 'foo' },
    { tag: 'p', children: ctx.bar, patchFlag: 1 /* 动态的 textContent */ },
  ]
}
```

有了这个信息，我们就可以在 `vnode` 的创建阶段把动态节点提取出来，什么样的节点是动态节点呢？带有 `patchFlag` 的节点就是动态节点，我们将它提取出来放到一个数组中存着，例如：

```
const vnode = {
  tag: 'div',
  children: [
    { tag: 'p', children: 'foo' },
    { tag: 'p', children: ctx.bar, patchFlag: 1 /* 动态的 textContent */ },
  ],
  dynamicChildren: [
    { tag: 'p', children: ctx.bar, patchFlag: 1 /* 动态的 textContent */ },
  ]
}
```

现在我们已经拿到了所有的动态节点，它们存储在 `dynamicChildren` 中，因此在 `diff` 过程中就可以避免按照 `vdom` 树一层一层的遍历，而是直接找到 `dynamicChildren` 进行更新。除了跳过无用的层级遍历之外，由于我们早早的就为 `vnode` 打上了 `patchFlag`，因此在更新 `dynamicChildren` 中的节点时，可以准确的知道需要为该节点应用哪些更新动作，这基本上就实现了靶向更新。

节点不稳定 - Block Tree

上面的这个 `div` 节点就是一个 `Block`，一个 `Block` 怎么也构不成 `Block Tree`，这就意味着在一颗 `vdom` 树中，会有多个 `vnode` 节点充当 `Block` 的角色，进而构成一颗 `Block Tree`。那么什么情况下一个 `vnode` 节点会充当 `block` 的角色呢？

来看下面这段模板：

```
<div>
  <section v-if="foo">
    <p>{{ a }}</p>
  </section>
  <div v-else>
    <p>{{ a }}</p>
  </div>
</div>
```

假设只要最外层的 `div` 标签是 `Block` 角色，那么当 `foo` 为真时，`block` 收集到的动态节点为：

```
cosnt block = {
  tag: 'div',
  dynamicChildren: [
    { tag: 'p', children: ctx.a, patchFlag: 1 }
  ]
}
```

当 `foo` 为假时，`block` 的内容如下：

```
cosnt block = {
  tag: 'div',
  dynamicChildren: [
    { tag: 'p', children: ctx.a, patchFlag: 1 }
  ]
}
```

可以发现无论 `foo` 为真还是假，`block` 的内容是不变的，这就意味什么在 `diff` 阶段不会做任何更新，但是我们也看到了：`v-if` 的是一个 `<section>` 标签，`v-else` 的是一个 `<div>` 标

签，所以这里就出问题了。实际上问题的本质在于 `dynamicChildren` 的 `diff` 是忽略 `vdom` 树层级的，如下模板也有同样的问题：

```
<div>
  <section v-if="foo">
    <p>{{ a }}</p>
  </section>
  <section v-else> <!-- 即使这里是 section -->
    <div> <!-- 这个 div 标签在 diff 过程中被忽略 -->
      <p>{{ a }}</p>
    </div>
  </section >
</div>
```

即使 `v-else` 的也是一个 `<section>` 标签，但由于前后 `DOM` 树的不稳定，也会导致问题。这时我们就思考，如何让 `DOM` 树的结构变稳定呢？

`v-if` 的元素作为 `Block`

如果让使用了 `v-if/v-else-if/v-else` 等指令的元素也作为 `Block` 会怎么样呢？我们拿如下模板为例：

```
<div>
  <section v-if="foo">
    <p>{{ a }}</p>
  </section>
  <section v-else> <!-- 即使这里是 section -->
    <div> <!-- 这个 div 标签在 diff 过程中被忽略 -->
      <p>{{ a }}</p>
    </div>
  </section>
</div>
```

如果我们让这两个 `section` 标签都作为 `block`，那么将构成一颗 `block tree`：

```
Block(Div)
    - Block(Section v-if)
    - Block(Section v-else)
```

父级 `Block` 除了会收集子代动态节点之外，也会收集子 `Block`，因此两个 `Block(section)` 将作为 `Block(div)` 的 `dynamicChildren`：

```
cosnt block = {
  tag: 'div',
  dynamicChildren: [
    { tag: 'section', { key: 0 }, dynamicChildren: [...]}, /* Block(Section v-if) */
    { tag: 'section', { key: 1 }, dynamicChildren: [...]}  /* Block(Section v-else) */]
}
```

这样当 `v-if` 条件为真时，`dynamicChildren` 中包含的是 `Block(section v-if)`，当条件为假时 `dynamicChildren` 中包含的是 `Block(section v-else)`，在 Diff 过程中，渲染器知道这是两个不同的 `Block`，因此会做完全的替换，这样就解决了 `DOM` 结构不稳定引起的问题。而这就是 `Block Tree`。

v-for 的元素作为 Block

不仅 `v-if` 会让 `DOM` 结构不稳定，`v-for` 也会，但是 `v-for` 的情况稍微复杂一些。思考如下模板：

```
<div>
  <p v-for="item in list">{{ item }}</p>
  <i>{{ foo }}</i>
  <i>{{ bar }}</i>
</div>
```

假设 list 值由 `[1 ,2]` 变为 `[1]`，按照之前的思路，最外层的 `<div>` 标签作为一个 `Block`，那么它更新前后对应的 `Block Tree` 应该是：

```
// 前
const prevBlock = {
  tag: 'div',
  dynamicChildren: [
    { tag: 'p', children: 1, 1 /* TEXT */ },
    { tag: 'p', children: 2, 1 /* TEXT */ },
    { tag: 'i', children: ctx.foo, 1 /* TEXT */ },
    { tag: 'i', children: ctx.bar, 1 /* TEXT */ },
  ]
```

```
}

// 后
const nextBlock = {
  tag: 'div',
  dynamicChildren: [
    { tag: 'p', children: item, 1 /* TEXT */ },
    { tag: 'i', children: ctx.foo, 1 /* TEXT */ },
    { tag: 'i', children: ctx.bar, 1 /* TEXT */ },
  ]
}

```

`prevBlcok` 中有四个动态节点，`nextBlock` 中有三个动态节点。这时候要如何进行 `Diff`？有的同学可能会说拿 `dynamicChildren` 进行传统 `Diff`，这是不对的，因为传统 `Diff` 的一个前置条件是同层级节点间的 `Diff`，但是 `dynamicChildren` 内的节点未必是同层级的。

实际上我们只需要让 `v-for` 的元素也作为一个 `Block` 就可以了。这样无论 `v-for` 怎么变化，它始终都是一个 `Block`，这保证了结构稳定，无论 `v-for` 怎么变化，这颗 `Block Tree` 看上去都是：

```
const block = {
  tag: 'div',
  dynamicChildren: [
    // 这是一个 Block 哦，它有 dynamicChildren
    { tag: Fragment, dynamicChildren: [/*.. v-for 的节点 ..*/] },
    { tag: 'i', children: ctx.foo, 1 /* TEXT */ },
    { tag: 'i', children: ctx.bar, 1 /* TEXT */ },
  ]
}
```

稳定的 Fragment

既然有不稳定的 `Fragment`，那就有稳定的 `Fragment`，什么样的 `Fragment` 是稳定的呢？

- v-for 的表达式是常量

```
<p v-for="n in 10"></p>

<!-- 或者 -->
<p v-for="s in 'abc'"></p>
```

由于 `10` 和 `'abc'` 是常量，所有这两个 `Fragment` 是不会变化的，因此它是稳定的，对于稳定的 `Fragment` 是不需要回退到传统 `Diff` 的，这在性能上会有一定的优势。

- 多个根元素

`Vue3` 不再限制组件的模板必须有一个根节点，对于多个根节点的模板，例如：

```
<template>
  <div>111</div>
  <p>222</p>
  <i>333</i>
</template>
```

如上，这也是一个稳定的 Fragment，有的同学或许会想如下模板也是稳定的 Fragment 吗：

```
<template>
  <div v-if="condition">111</div>
  <p>222</p>
  <i>333</i>
</template>
```

这其实也是稳定的，因为带有 `v-if` 指令的元素本身作为 `Block` 存在，所以这段模板的 `Block Tree` 结构总是：

```
Block(Fragment)
    - Block(div v-if)
    - VNode(p)
    - VNode(i)
```

对应到 `VNode` 应该类似于：

```
const block = {
  tag: Fragment,
  dynamicChildren: [
    { tag: 'div', dynamicChildren: [...] },
    { tag: 'p' },
    { tag: 'i' },
  ],
  PatchFlags.STABLE_FRAGMENT
}
```

无论如何，它的结构都是稳定的。需要注意的是这里的 `PatchFlags.STABLE_FRAGMENT` ，该标志必须存在，否则会回退传统 `Diff` 模式。

2. 静态提升

- 提升静态节点树

`Vue3` 的 `Compiler` 如果开启了 `hoistStatic` 选项则会提升静态节点，或静态的属性，这可以减少创建 `VNode` 的消耗，如下模板所示：

```
<div>
  <p>text</p>
</div>
```

在没有被提升的情况下其渲染函数相当于：

```
function render() {
  return (openBlock(), createBlock('div', null, [
    createVNode('p', null, 'text')
  ]))
}
```

很明显， `p` 标签是静态的，它不会改变。但是如上渲染函数的问题也很明显，如果组件内存在动态的内容，当渲染函数重新执行时，即使 `p` 标签是静态的，那么它对应的 `VNode` 也会重新创建。当开启静态提升后，其渲染函数如下：

```
const hoist1 = createVNode('p', null, 'text')

function render() {
  return (openBlock(), createBlock('div', null, [
    hoist1
  ]))
}
```

这就实现了减少 `VNode` 创建的性能消耗。需要了解的是，静态提升是以树为单位的，如下模板所示：

```
<div>
  <section>
```

```
    <p>
      <span>abc</span>
    </p>
  </section>
</div>
```

除了根节点的 `div` 作为 block 不可被提升之外，整个 `<section>` 元素及其子代节点都会被提升，因为他们是整棵树都是静态的。如果我们把上面代码中的 `abc` 换成 `{{ abc }}`，那么整棵树都不会被提升。再看如下代码：

- 元素不会被提升的情况
  - 元素带有动态的 `key` 绑定

  除了刚刚讲到的元素的所有子代节点必须都是静态的才会被提升之外还有哪些情况下会阻止提升呢？

  如果一个元素有动态的 `key` 绑定那么它是不会被提升的，例如：

```
<div :key="foo"></div>
```

  实际上一个元素拥有任何动态绑定都不应该被提升，那么为什么 `key` 会被单独拿出来？实际上 `key` 和普通的 `props` 相比，它对于 `VNode` 的意义是不一样的，普通的 `props` 如果它是动态的，那么只需要体现在 `PatchFlags` 上就可以了，例如：

```
<div>
  <p :foo="bar"></p>
</div>
```

  我们可以为 `p` 标签打上 `PatchFlags`：

```
render(ctx) {
  return (openBlock(), createBlock('div', null, [
    createVNode('p', { foo: ctx }, null, PatchFlags.PROPS, ['foo'])
  ]))
}
```

  注意到在创建 `VNode` 时，为其打上了 `PatchFlags.PROPS`，代表这个元素需要更新 `PROPS`，并且需要更新的 `PROPS` 的名字叫 `foo`。

`key` 本身具有特殊意义，它是 `VNode` (或元素) 的唯一标识，即使两个元素除了 `key` 以外一切都相同，但这两个元素仍然是不同的元素，对于不同的元素需要做完全的替换处理才行，而 `PatchFlags` 用于在同一个元素上的属性补丁，因此 `key` 是不同于其它 `props` 的。

正因为 `key` 的值是动态的可变的，因此对于拥有动态 `key` 的元素，它始终都应该参与到 `diff` 中并且不能简单的打 `PatchFlags` 补丁标识，那应该怎么做呢？很简单，让拥有动态 `key` 的元素也作为 `Block` 即可，以如下模板为例：

```
<div>
  <div :key="foo"></div>
</div>
```

它对应的渲染函数应该是：

```
render(ctx) {
  return (openBlock(), createBlock('div', null, [
    (openBlock(), createBlock('div', { key: ctx.foo }))
  ]))
}
```

- 使用 `ref` 的元素

如果一个元素使用了 `ref` ，无论是否动态绑定的值，那么这个元素都不会被静态提升，这是因为在每一次 `patch` 时都需要设置 `ref` 的值，如下模板所示：

```
<div ref="domRef"></div>
```

乍一看觉得这完全就是一个静态元素，没错，元素本身不会发生变化，但由于 `ref` 的特性，导致我们必须在每次 `Diff` 的过程中重新设置 `ref` 的值，为什么呢？来看一个使用 `ref` 的场景：

```
<template>
  <div>
    <p ref="domRef"></p>
  </div>
</template>

<script>
export default {
  setup() {
    const refP1 = ref(null)
```

```
      const refP2 = ref(null)
      const useP1 = ref(true)

      return {
        domRef: useP1 ? refP1 : refP2
      }
    }
  }
</script>
```

如上代码所示，`p` 标签使用了一个非动态的 `ref` 属性，值为字符串 `domRef`，同时我们注意到 `setupContext`(我们把 `setup` 函数返回的对象叫做 `setupContext`)中也包含了同名的 `domRef` 属性，这不是偶然，他们之间会建立联系，最终结果就是：

- 当 `useP1` 为真时，`refP1.value` 引用 `p` 元素
- 当 `useP1` 为假时，`refP2.value` 引用 `p` 元素

因此，即使 `ref` 是静态的，但很显然在更新的过程中由于 `useP1` 的变化，我们不得不更新 `domRef`，所以只要一个元素使用了 `ref`，它就不会被静态提升，并且这个元素对应的 `VNode` 也会被收集到父 `Block` 的 `dynamicChildren` 中。

但由于 `p` 标签除了需要更新 `ref` 之外，并不需要更新其他 `props`，所以在真实的渲染函数中，会为它打上一个特殊的 `PatchFlag`，叫做：`PatchFlags.NEED_PATCH`：

```
render() {
  return (openBlock(), createBlock('div', null, [
    createVNode('p', { ref: 'domRef' }, null, PatchFlags.NEED_PATCH)
  ]))
}
```

- 使用自定义指令的元素

实际上一个元素如果使用除 `v-pre/v-cloak` 之外的所有 `Vue` 原生提供的指令，都不会被提升，使用自定义指令也不会被提升，例如：

```
<p v-custom></p>
```

和使用 `key` 一样，会为这段模板对应的 `VNode` 打上 `NEED_PATCH` 标志。

- 提升静态 `PROPS`

前面说过，静态节点的提升以树为单位，如果一个 `VNode` 存在非静态的子代节点，那么该 `VNode` 就不是静态的，也就不会被提升。但这个 `VNode` 的 `props` 却可能是静态的，这使我们可以将它

的 `props` 进行提升，这同样可以节约 `VNode` 对象的创建开销，内存占用等，例如：

```
<div>
  <p foo="bar" a="b">{{ text }}</p>
</div>
```

在这段模板中 `p` 标签有动态的文本内容，因此不可以被提升，但 `p` 标签的所有属性都是静态的，因此可以提升它的属性，经过提升后其渲染函数如下：

```
const hoistProp = { foo: 'bar', a: 'b' }

render(ctx) {
  return (openBlock(), createBlock('div', null, [
    createVNode('p', hoistProp, ctx.text)
  ]))
}
```

3. 预字符串化

静态提升的 `VNode` 节点或节点树本身是静态的，那么能否将其预先字符串化呢？如下模板所示：

```
<div>
  <p></p>
  <p></p>
  ...20 个 p 标签
  <p></p>
</div>
```

假设如上模板中有大量连续的静态的 `p` 标签，当采用了 `hoist` 优化时，结果如下：

```
cosnt hoist1 = createVNode('p', null, null, PatchFlags.HOISTED)
cosnt hoist2 = createVNode('p', null, null, PatchFlags.HOISTED)
... 20 个 hoistx 变量
cosnt hoist20 = createVNode('p', null, null, PatchFlags.HOISTED)

render() {
  return (openBlock(), createBlock('div', null, [
    hoist1, hoist2, ...20 个变量, hoist20
  ]))
}
```

预字符串化会将这些静态节点序列化为字符串并生成一个 `Static` 类型的 `VNode`：

```
const hoistStatic = createStaticVNode('<p></p><p></p><p></p>...20个...<p></p>')

render() {
  return (openBlock(), createBlock('div', null, [
    hoistStatic
  ]))
}
```

这有几个明显的优势：

- 生成代码的体积减少
- 减少创建 VNode 的开销
- 减少内存占用

静态节点在运行时会通过 `innerHTML` 来创建真实节点，因此并非所有静态节点都是可以预字符串化的，可以预字符串化的静态节点需要满足以下条件：

- 非表格类标签：caption 、thead、tr、th、tbody、td、tfoot、colgroup、col
- 标签的属性必须是：
  - 标准 HTML attribute： https://developer.mozilla.org/en-US/docs/Web/HTML/Attributes
  - 或 data-/aria- 类属性

当一个节点满足这些条件时代表这个节点是可以预字符串化的，但是如果只有一个节点，那么并不会将其字符串化，可字符串化的节点必须连续且达到一定数量才行：

- 如果节点没有属性，那么必须有连续 20 个及以上的静态节点存在才行，例如：

```
<div>
  <p></p>
  <p></p>
  ... 20 个 p 标签
  <p></p>
</div>
```

或者在这些连续的节点中有 5 个及以上的节点是有属性绑定的节点：

```
<div>
```

```
  <p id="a"></p>
  <p id="b"></p>
  <p id="c"></p>
  <p id="d"></p>
  <p id="e"></p>
</div>
```

这段节点的数量虽然没有达到 20 个，但是满足 5 个节点有属性绑定。

这些节点不一定是兄弟关系，父子关系也是可以的，只要阈值满足条件即可，例如：

```
<div>
  <p id="a">
    <p id="b">
      <p id="c">
        <p id="d">
          <p id="e">
          </p>
        </p>
      </p>
    </p>
  </p>
</div>
```

预字符串化会在编译时计算属性的值，例如：

```
<div>
  <p :id="'id-' + 1">
    <p :id="'id-' + 2">
      <p :id="'id-' + 3">
        <p :id="'id-' + 4">
          <p :id="'id-' + 5"></p>
        </p>
      </p>
    </p>
  </p>
</div>
```

4. Cache Event handler

如下组件的模板所示：

```
<Comp @change="a + b" />
```

这段模板如果手写渲染函数的话相当于：

```
render(ctx) {
  return h(Comp, {
    onChange: () => (ctx.a + ctx.b)
  })
}
```

很显然，每次 `render` 函数执行的时候，`Comp` 组件的 `props` 都是新的对象，`onChange` 也会是全新的函数。这会导致触发 `Comp` 组件的更新。

当 Vue3 Compiler 开启 `prefixIdentifiers` 以及 `cacheHandlers` 时，这段模板会被编译为：

```
render(ctx, cache) {
  return h(Comp, {
    onChange: cache[0] || (cache[0] = ($event) => (ctx.a + ctx.b))
  })
}
```

这样即使多次调用渲染函数也不会触发 `Comp` 组件的更新，因为 `Vue` 在 `patch` 阶段比对 `props` 时就会发现 `onChange` 的引用没变。

5. v-once

这是 `Vue2` 就支持的功能，`v-once` 是一个“很指令”的指令，因为它就是给编译器看的，当编译器遇到 `v-once` 时，会利用我们刚刚讲过的 `cache` 来缓存全部或者一部分渲染函数的执行结果，例如如下模板：

```
<div>
  <div v-once>{{ foo }}</div>
</div>
```

会被编译为：

```
render(ctx, cache) {
  return (openBlock(), createBlock('div', null, [
```

```
    cache[1] || (cache[1] = h("div", null, ctx.foo, 1 /* TEXT */))
  ]))
}
```

这样就缓存了这段 `vnode`。既然 `vnode` 已经被缓存了，后续的更新就都会读取缓存的内容，而不会重新创建 `vnode` 对象了，同时在 Diff 的过程中也就不需要这段 vnode 参与了，因此你通常会看到编译后的代码更接近如下内容：

```
render(ctx, cache) {
  return (openBlock(), createBlock('div', null, [
    cache[1] || (
      setBlockTracking(-1), // 阻止这段 VNode 被 Block 收集
      cache[1] = h("div", null, ctx.foo, 1 /* TEXT */),
      setBlockTracking(1), // 恢复
      cache[1] // 整个表达式的值
    )
  ]))
}
```

稍微解释一下这段代码，我们已经讲解过何为 “Block Tree”，而 `openBlock()` 和 `createBlock()` 函数用来创建一个 `Block`。而 `setBlockTracking(-1)` 则用来暂停收集的动作，所以在 `v-once` 编译生成的代码中你会看到它，这样使用 `v-once` 包裹的内容就不会被收集到父 `Block` 中，也就不参与 `Diff` 了。

所以，`v-once` 带来的性能提升来自两方面：

- 1、VNode 的创建开销
- 2、无用的 Diff 开销